

ERP IA UI-DRIVEN

Architecture, Stack & Feuille de Route

Document de référence technique

Architecture Multi-Tenant | RBAC/ABAC | IA Conversationnelle

Avril 2026

1. Vision & Concept

Cet ERP repose sur trois piliers fondamentaux qui le différencient des solutions classiques :

Pilier 1 — UI-Driven par l'IA

Le backend génère dynamiquement la structure de l'interface selon le rôle, le contexte et les permissions de chaque utilisateur.

Flutter reçoit un JSON schema et rend les composants sans logique propre — c'est le Backend-Driven UI (BDUI).

L'IA adapte l'affichage en temps réel selon les actions de l'utilisateur.

Pilier 2 — Configuration conversationnelle

Un nouveau client configure son ERP via un chat IA (30-45 min) sans écrans de paramétrage complexes.

L'IA extrait les entités métier, les rôles, les workflows et les règles depuis la conversation.

La config est stockée par tenant en JSON et appliquée immédiatement.

Pilier 3 — RBAC + ABAC natif

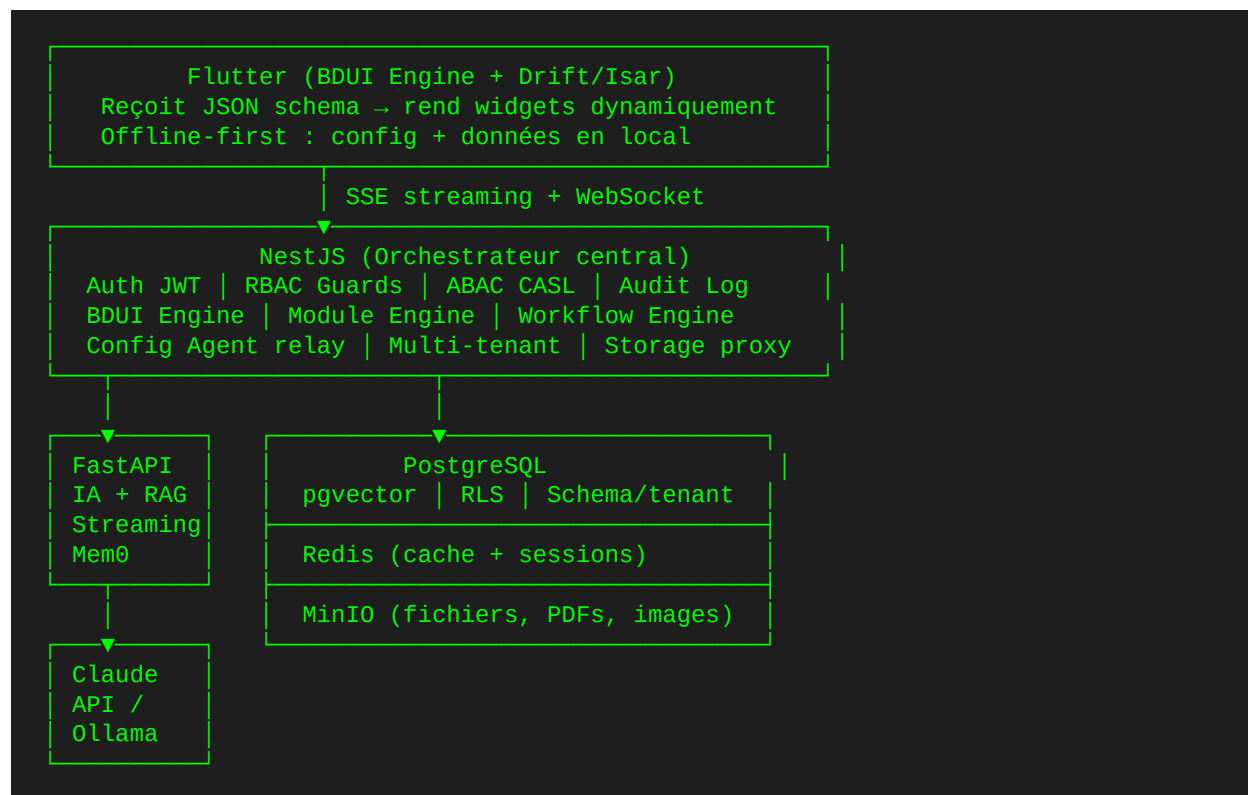
Chaque utilisateur a un rôle (RBAC) ET des attributs contextuels (ABAC : département, plafond, horaires).

Les permissions filtrent aussi bien l'UI que les données remontées au LLM — le modèle IA ne voit jamais des données non autorisées.

Row-Level Security PostgreSQL en dernière ligne de défense.

2. Architecture Technique

2.1 Vue d'ensemble



2.2 Décision stack — NestJS seul, pas Supabase

Pourquoi NestJS seul suffit

Supabase apporte : Auth, Realtime, Storage, Dashboard DB, PostgREST.

NestJS couvre déjà : Auth (Passport.js + JWT), Realtime (WebSocket Gateway), Storage (via MinIO).

Supabase self-hosted = 8 services Docker supplémentaires pour une valeur marginale.

MinIO remplace le Storage Supabase en 1 seul service léger et S3-compatible.

Adminer ou pgAdmin remplace le Studio Supabase pour visualiser la DB en dev.

Verdict : stack plus simple, contrôle total, moins de services à maintenir.

2.3 Stack complète — 5 services Docker

```
# docker-compose.yml
services:
```

```

nestjs:      # API principal – auth, BDUI, modules, workflows
fastapi:     # Microservice IA – RAG, streaming, Config Agent
postgresql:  # Données + pgvector + RLS + schema-per-tenant
redis:       # Cache config, sessions, rate limiting LLM
minio:       # Storage fichiers – PDFs, ordonnances, images

```

```

# Dev seulement (pas en production) :
adminer:     # Dashboard visuel DB – remplace Supabase Studio

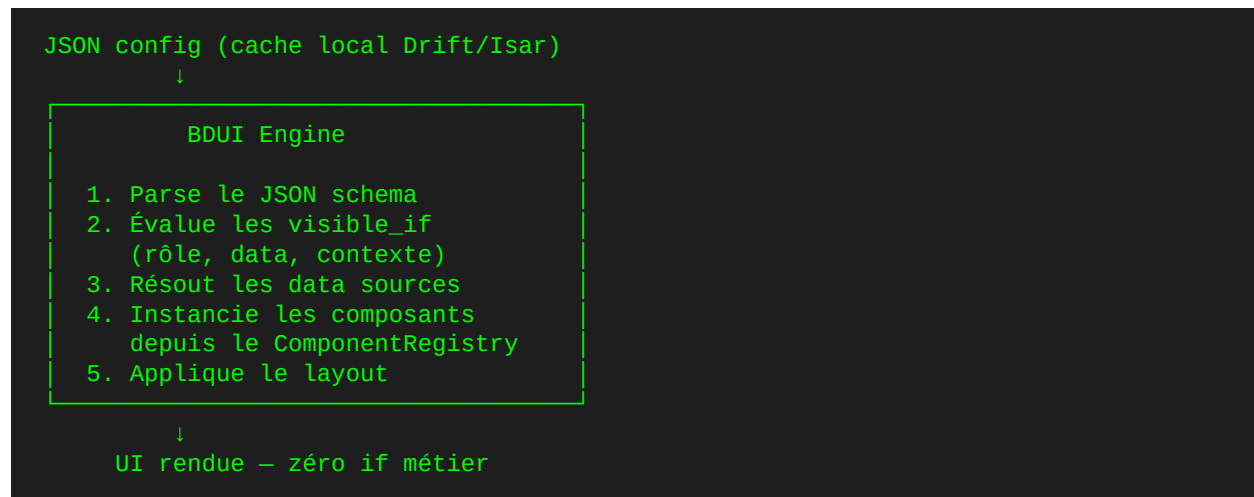
```

Couche	Technologie	Rôle
App mobile/desktop	Flutter + shadcn_ui + Drift/Isar	BDUI renderer + offline-first
Admin web	React + ShadCN + Next.js	Config chat + preview BDUI
API principal	NestJS	Auth, RBAC/ABAC, BDUI, Modules, Workflows
Auth	Passport.js + @nestjs/jwt	JWT, refresh tokens, OAuth2 — dans NestJS
Realtime	NestJS WebSocket Gateway	Streaming IA, notifications — dans NestJS
IA Inférence	FastAPI + LangChain	RAG, Config Agent, streaming LLM
Base de données	PostgreSQL + pgvector	Données + RAG vectoriel + RLS
Cache / Sessions	Redis	Sessions, config cache, rate limiting LLM
Storage fichiers	MinIO (S3-compatible)	PDFs, ordonnances, images — remplace Supabase Storage
Mémoire IA	Mem0	Contexte long terme par utilisateur
Parsing documents	Docling	PDF, Excel, Word → chunks indexables
RAG	LlamaIndex + pgvector	Indexation + hybrid search données métier
Permissions	CASL + Casbin	ABAC complexe dans NestJS
LLM	Claude API / Ollama	Cloud ou local selon connectivité client
Composants UI	shadcn_ui Flutter + ShadCN React	Design System cohérent web + mobile
Maquettage	Widgetbook (Flutter)	Storybook vivant des composants BDUI
DB Dashboard	Adminer (dev)	Visualisation DB — léger, 1 service Docker

3. Backend-Driven UI (BDUI) — Engine & Architecture

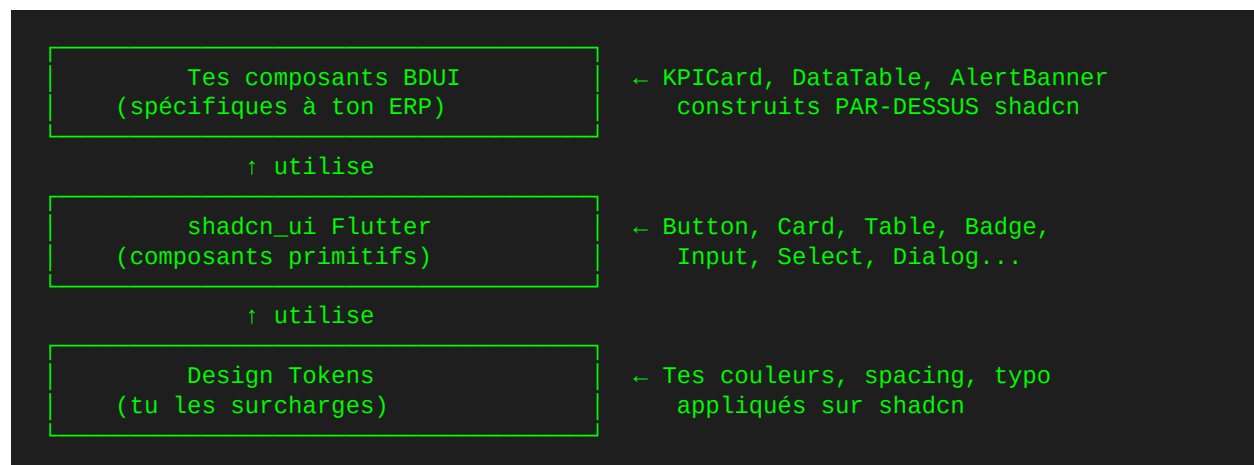
3.1 Principe fondamental

Le serveur décide quoi afficher. Flutter ne contient aucun if métier — il est un moteur de rendu qui lit des règles JSON. NestJS génère le JSON schema selon l'utilisateur, son rôle, son contexte et la config du tenant.



3.2 shadcn_ui — la base des composants

shadcn_ui existe sur Flutter ET sur React. Même langage visuel des 2 côtés : l'app mobile et la preview web de l'interface admin sont visuellement cohérentes. Tu construis TES composants BDUI métier par-dessus shadcn_ui — tu ne pars pas de zéro.



3.3 Le ComponentRegistry — coeur de l'Engine

```
// Le registre – tous les composants disponibles
class ComponentRegistry {
    static final Map<String, ComponentBuilder> _builders = {
        'KPICard':    (config, ctx) => KPICard.fromConfig(config, ctx),
        'DataTable':  (config, ctx) => ERPDataTable.fromConfig(config, ctx),
        'AlertBanner': (config, ctx) => AlertBanner.fromConfig(config, ctx),
        'FAB':        (config, ctx) => ERPFAB.fromConfig(config, ctx),
        'ListTile':   (config, ctx) => ERPListTile.fromConfig(config, ctx),
        'ChartBar':   (config, ctx) => ChartBar.fromConfig(config, ctx),
        'FormSection': (config, ctx) => FormSection.fromConfig(config, ctx),
        // tu ajoutes ici au fur et à mesure
    };

    static Widget build(ComponentConfig config, BuildContext ctx) {
        final builder = _builders[config.type];
        if (builder == null) return UnknownComponent(config.type);
        return builder(config, ctx);
    }
}
```

3.4 RuleEvaluator — les if sont dans le JSON

```
// ❌ JAMAIS ça dans Flutter :
if (user.role == 'MANAGER' && commande.montant > 500000) {
    showValidationButton();
}

// ✅ Flutter lit la règle depuis le JSON config :
class RuleEvaluator {
    bool evaluate(Rule? rule) {
        if (rule == null) return true;
        return switch (rule.operator) {
            'AND' => rule.children!.every(evaluate),
            'OR'  => rule.children!.any(evaluate),
            'role' => user.roles.contains(rule.value),
            '>'   => resolveField(rule.field!) > rule.value,
            '<'   => resolveField(rule.field!) < rule.value,
            '=='  => resolveField(rule.field!) == rule.value,
            _     => true,
        };
    }
}

// La règle dans le JSON :
{ "visible_if": { "AND": [
    { "role": ["MANAGER", "DG"] },
    { "field": "montant", "operator": ">", "value": 500000 }
]}}
```

3.5 Les Layouts — fixes et prédéfinis

Tu n'as que 4-5 layouts max, tous codés une fois. L'IA choisit lequel utiliser — elle ne crée pas de nouveaux layouts.

```
class LayoutResolver {
  static Widget build({ required String layout,
                      required Map<String, List<Widget>> zones }) {
    return switch (layout) {
      'dashboard' => DashboardLayout(zones: zones),
      'list'      => ListLayout(zones: zones),
      'form'      => FormLayout(zones: zones),
      'detail'    => DetailLayout(zones: zones),
      _          => DashboardLayout(zones: zones),
    };
  }
}

// DashboardLayout – codé une fois, toujours beau
// Zone kpis → toujours en haut (GridView 2 cols)
// Zone main → contenu principal
// Zone actions → FAB toujours en bas à droite
```

3.6 Avantages pour l'ERP

- Zéro if métier dans le code Flutter — toute la logique est dans le JSON
- L'UI évolue sans mise à jour App Store — changer la config suffit
- Le même Flutter rend n'importe quel module ERP de n'importe quel client
- Offline-first natif — la config EST l'app, stockée dans Drift/Isar
- Multi-tenant : chaque client a son UI personnalisée depuis sa config
- Preview web identique grâce à ShadCN React + même JSON Schema

3b. Maquettage BDUI — Comment faire sans écrans fixes

3b.1 Le problème du maquettage classique

ERP classique	Ton ERP BDUI
Tu maquettes écran par écran	Tu ne peux pas maquetter des écrans fixes
Figma page 1 = écran Dashboard	Les écrans n'existent pas — ils sont générés
Tu codes écran par écran	Tu codes un engine + des composants
Modifier = retoucher le code	Modifier = changer le JSON de config

3b.2 Ce que tu maquettes vraiment

Tu maquettes des composants isolés et des layouts — pas des écrans. Ensuite tu valides avec des 'compositions exemples' qui montrent comment ils s'assemblent.

Structure de maquettage (Figma ou Widgetbook) :

```
— Tokens
  — Couleurs (primary, success, danger, surface)
  — Spacing (4, 8, 12, 16, 24, 32, 48, 64)
  — Typographie (h1, h2, body, caption)

— Composants/ (chaque composant avec tous ses états)
  — KPICard (normal, warning, danger, loading)
  — DataTable (vide, chargement, données, erreur)
  — AlertBanner (info, success, warning, danger)
  — ListTile (simple, avec avatar, avec badge)
  — FormSection (texte, select, date, switch)

— Layouts/
  — dashboard_layout (zones : top, main, aside, fab)
  — list_layout
  — form_layout

— Compositions exemples/ (pour valider l'assemblage)
  — Exemple Pharmacie Dashboard
  — Exemple BTP Liste Chantiers
```

3b.3 Widgetbook — ton Figma vivant dans Flutter

Widgetbook (ou flutter_storybook) te permet de développer et visualiser chaque composant isolément avec tous ses états — directement dans Flutter. C'est ton outil de maquettage principal, pas Figma.

```
// Widgetbook — chaque composant avec ses variations
WidgetbookComponent(
  name: 'KPICard',
  useCases: [
    WidgetbookUseCase(
      name: 'Normal',
      builder: (_) => KPICard(
        value: '142', label: 'Commandes', trend: '+12%'
      )
    ),
    WidgetbookUseCase(
      name: 'Warning',
      builder: (_) => KPICard(
        value: '3', label: 'Ruptures', status: Status.warning
      )
    ),
    WidgetbookUseCase(
      name: 'Danger',
      builder: (_) => KPICard(
        value: '0', label: 'Stock', status: Status.danger
      )
    ),
  ],
)
]
```

3b.4 Sandbox JSON — valider l'Engine

Un écran de dev qui charge un JSON local et te montre le résultat rendu par l'Engine. C'est ta maquette vivante qui prouve que le BDUI fonctionne bout en bout.

```
// assets/sandbox/pharmacie_dashboard.json
{
  "screen": "dashboard",
  "layout": "dashboard",
  "zones": {
    "kpis": [
      { "type": "KPICard", "props": { "label": "Ventes jour",
        "source": "ventes.total_today", "format": "currency_xof" } },
      { "type": "KPICard", "props": { "label": "Ruptures",
        "source": "stock.ruptures", "status": "danger" } }
    ],
    "main": [
      { "type": "DataTable", "props": {
        "source": "commandes.recent",
```

```

    "columns": ["ref", "client", "montant", "statut"] } }
  ],
  "actions": [
    { "type": "FAB", "props": { "action": "create_commande" },
      "visible_if": { "role": ["VENDEUR", "MANAGER"] } }
  ]
}
}

// Flutter charge ce JSON → Engine le rend
// Tu vois exactement ce que le client verra

```

3b.5 Ordre de développement recommandé

Semaine	Tâche	Livrable
Semaine 1	JSON Schema BDUI (ComponentConfig, ScreenConfig, Rule)	Contrat TypeScript/Dart validé
Semaine 2	shadcn_ui Flutter + Design Tokens	Theming appliqué, primitifs disponibles
Semaine 3	Widgetbook + 5 composants de base	KPICard, DataTable, AlertBanner, ListTile, FAB
Semaine 4	BDUI Engine (Registry, RuleEvaluator, 2 layouts)	Sandbox JSON → rendu fonctionnel
Semaine 5-6	FormSection, ChartBar + JSON configs exemples	Pharmacie et BTP rendus depuis JSON
Mois 2	Admin web ShadCN React + preview	Chat config + preview identique à Flutter

4. RBAC + ABAC — Sécurité Multi-Couche

4.1 Différence RBAC vs ABAC

RBAC	ABAC
Rôle → Permission simple	User + Ressource + Contexte → Décision
Ex: MANAGER voit les factures	Ex: MANAGER voit les factures DE SON DEPT si montant < 500k XOF
Simple à implémenter	Granulaire, plus complexe
NestJS Guards natifs	CASL / Casbin + règles dynamiques

4.2 Architecture des couches de sécurité

```
Requête utilisateur
  ↓ Layer 1 : JWT Guard (NestJS) — authentication
  ↓ Layer 2 : RBAC Guard — le rôle a-t-il accès à cette route ?
  ↓ Layer 3 : ABAC CASL — les attributs autorisent-ils cet objet ?
  ↓ Layer 4 : pgvector filter — RAG filtrée par dept/rôle
  ↓ Layer 5 : PostgreSQL RLS — sécurité au niveau base de données
  → Données retournées (déjà filtrées avant d'atteindre le LLM)
```

4.3 Règle critique avec l'IA

Ne jamais laisser le LLM décider des permissions

FAUX : Envoyer toutes les données au LLM et lui dire 'montre seulement ce que l'user peut voir'
CORRECT : Filtrer les données AVANT d'appeler le LLM — le modèle ne reçoit que les données autorisées

Le LLM est un moteur de traitement, pas un garde de sécurité.

Toujours auditer chaque query LLM dans une table dédiée (qui a demandé quoi, quand, avec quelles données).

5. Configuration Conversationnelle par IA

5.1 Le flux d'onboarding

```

Étape 1 – Découverte
  IA: "Quel est votre secteur ?"
  User: "Pharmacie en gros à Ouagadougou"

Étape 2 – Rôles & équipe
  IA: "Combien d'employés et quels rôles ?"
  User: "12 : vendeurs, livreurs, comptable, DG"

Étape 3 – Workflows
  IA: "Comment se passe une commande ?"
  User: "Commande → vérif stock → validation si >500k → livraison → facture"

Étape 4 – Génération
  IA extrait → JSON config → validation → déploiement
  IA: "Voici votre ERP configuré. Souhaitez-vous ajuster ?"

```

5.2 Ce que l'IA extrait automatiquement

Élément extrait	Exemple input	Exemple output JSON
Entités métier	"on gère des bons de livraison"	{ "entite": "BonLivraison", "champs": [...] }
Rôles RBAC	"vendeurs, comptable, DG"	{ "roles": ["VENDEUR", "COMPTABLE", "DG"] }
Règles ABAC	"DG valide si >500k"	{ "condition": "montant>500000", "approver": "DG" }
Modules ERP	"stock, commandes, factures"	{ "modules": ["stock", "commandes", "facturation"] }
Workflow steps	"vérif stock → validation → livraison"	{ "steps": ["check_stock", "approve", "deliver"] }

5.3 Multi-tenant isolé

Chaque client possède sa propre configuration stockée en JSONB dans PostgreSQL. Un seul codebase NestJS + Flutter sert tous les clients avec des expériences complètement différentes.

Client A — Pharmacie	Client B — BTP
Modules: stock, ordonnances, facturation	Modules: chantiers, devis, ouvriers, matériaux

Workflow: prescription → vérif → vente	Workflow: devis → validation DG → bon chantier
UI: orientée caisse et stock	UI: orientée terrain et ressources
Rôles: pharmacien, caissier, DG	Rôles: chef chantier, ouvrier, comptable, DG

6. Compétences à Maîtriser

6.1 Frontend — Flutter

Compétence	Niveau requis	Pourquoi
Dart avancé (async, streams)	Expert	BDUI + SSE streaming temps réel
State management (Riverpod/Bloc)	Expert	UI dynamique complexe
Drift / Isar (SQLite)	Avancé	Offline-first, sync queue
JSON Schema → Widgets (BDUI Engine)	Expert	Coeur du renderer — ComponentRegistry, RuleEvaluator, LayoutResolver
shadcn_ui Flutter	Avancé	Composants primitifs du Design System
Widgetbook	Intermédiaire	Maquettage vivant des composants BDUI
WebSockets + SSE	Avancé	Streaming réponses IA
Animations (Rive, Lottie)	Intermédiaire	Transitions BDUI fluides

6.2 Backend — NestJS

Compétence	Niveau requis	Pourquoi
NestJS modules/guards/interceptors	Expert	Architecture ERP modulaire
CASL / Casbin	Avancé	ABAC complexe
JWT + OAuth2	Avancé	Auth multi-tenant
TypeORM / Prisma	Avancé	Modèles de données ERP
Microservices NestJS	Avancé	Communication avec FastAPI
Bull / BullMQ (job queues)	Avancé	Tâches asynchrones, sync offline
SSE / WebSocket Gateway	Avancé	Streaming IA vers Flutter

6.3 IA & Data Layer

Compétence	Niveau requis	Pourquoi
------------	---------------	----------

LlamaIndex (RAG)	Avancé	Indexation données métier ERP
LangChain Agents	Avancé	Config conversationnelle
pgvector + embeddings	Avancé	Recherche sémantique filtrée
Prompt engineering	Expert	Extraction structurée fiable
Mem0 / Zep	Intermédiaire	Mémoire utilisateur long terme
Structured outputs (Instructor)	Avancé	JSON config depuis conversation
Streaming SSE depuis LLM	Avancé	UX temps réel

6.4 Infrastructure & DevOps

Compétence	Niveau requis	Pourquoi
PostgreSQL avancé (RLS, JSONB)	Expert	Multi-tenant + sécurité
Redis (cache, pub/sub)	Avancé	Sessions, rate limiting LLM
Docker + Docker Compose	Avancé	Isolation des services
CI/CD (GitHub Actions)	Intermédiaire	Déploiement continu
Observabilité (Langfuse)	Intermédiaire	Traçage des appels LLM
Ollama	Intermédiaire	LLM local pour clients offline

7. Niveau Algorithmique — Ce qui fait la Différence

C'est ici que se joue la robustesse que les autres n'ont pas. La plupart des ERP IA s'arrêtent au RAG basique. Ces algorithmes te donnent une longueur d'avance significative.

7.1 Graphe de Workflow (indispensable)

Algorithme : Directed Acyclic Graph (DAG) pour les workflows

Niveau : Intermédiaire-Avancé

Pourquoi : Un workflow ERP est un graphe orienté (Commande → Stock → Validation → Livraison).

Ce qu'il faut maîtriser : Topological Sort (Kahn's algorithm), détection de cycles, BFS/DFS.

Application : Valider qu'un workflow généré par l'IA est cohérent (pas de cycles, étapes atteignables).

Bibliothèque : graphlib (JS), networkx (Python)

```
// Exemple : validation d'un workflow généré par l'IA
function validateWorkflow(steps: WorkflowStep[]): boolean {
  const graph = buildDAG(steps);
  if (hasCycle(graph)) throw new Error('Workflow circulaire détecté');
  const sorted = topologicalSort(graph); // Kahn's algorithm
  return checkAllStepsReachable(sorted, steps);
}
```

7.2 Synchronisation Offline (CRDT)

Algorithme : Conflict-free Replicated Data Types (CRDT)

Niveau : Avancé

Pourquoi : Quand un utilisateur modifie des données offline sur Flutter et qu'un autre modifie les mêmes données en ligne, qui gagne ?

Ce qu'il faut maîtriser : LWW-Register (Last-Write-Wins), Vector Clocks, Operational Transform.

Application : Merge automatique des données offline sans conflit ni perte.

Alternative simple pour commencer : Timestamp + server-wins avec conflict queue pour les cas ambigus.

```
// Stratégie CRDT simplifiée pour l'ERP
interface CRDTRecord {
  id: string;
  vectorClock: Record<string, number>; // { userId: lamportTimestamp }
  value: any;
  lastModifiedBy: string;
```

```

}

function merge(local: CRDTRecord, remote: CRDTRecord): CRDTRecord {
  // Compare vector clocks → détermine le gagnant
  // En cas d'égalité → conflict queue pour résolution manuelle ou IA
}

```

7.3 Moteur de Règles Métier (Rete Algorithm)

Algorithme : Rete Algorithm pour les règles ABAC complexes

Niveau : Avancé

Pourquoi : Évaluer des centaines de règles ABAC pour des milliers d'utilisateurs simultanés naïvement est $O(n \times m)$ — trop lent.

Rete précalcule un réseau de patterns et réduit drastiquement le nombre d'évaluations.

Application : 'Cet utilisateur peut-il approuver cette commande ?' évalué en $O(1)$ après compilation.

Bibliothèque : Nools (JS), Drools (Java), ou implémentation custom avec NestJS

7.4 RAG avec Reranking (au-delà du RAG basique)

Algorithme : Hybrid Search + Cross-Encoder Reranking

Niveau : Avancé

Pourquoi : Le RAG basique (embedding similarity) rate des résultats pertinents. Le hybrid search combine vecteur + BM25 (keyword). Le reranking classe ensuite les résultats avec un cross-encoder.

Résultat : 30-40% de pertinence en plus sur les données ERP métier.

Stack : pgvector (vecteur) + pg_trgm (BM25-like) + Cohere Rerank ou sentence-transformers

```

// Hybrid search : vecteur + keyword
async function hybridSearch(query: string, userId: string) {
  const embedding = await embed(query);
  const userFilter = { departementId: user.deptId }; // ABAC filter

  const [vectorResults, keywordResults] = await Promise.all([
    pgvector.query(embedding, { filter: userFilter, limit: 20 }),
    pgFullText.search(query, { filter: userFilter, limit: 20 })
  ]);

  const merged = reciprocalRankFusion(vectorResults, keywordResults);
  return reranker.rerank(query, merged, { topN: 5 }); // Cross-encoder
}

```

7.5 State Machine pour les Workflows (XState)

Algorithme : Finite State Machine (FSM) pour les statuts ERP

Niveau : Intermédiaire

Pourquoi : Une commande ERP passe par des états (brouillon → soumise → validée → livrée → facturée). Une FSM garantit qu'une transition illégale est impossible.

Application : Générer automatiquement les FSM depuis la config conversationnelle. L'IA décrit le workflow, XState le modélise, NestJS l'exécute.

Bibliothèque : XState (NestJS), statemachine (Python)

7.6 Extraction Structurée (NLP Interne)

Algorithme : Named Entity Recognition (NER) guidé par schema

Niveau : Intermédiaire-Avancé

Pourquoi : Pour la config conversationnelle, il faut extraire des entités métier de manière fiable depuis du texte non structuré.

Approche : Instructor (Python) force le LLM à retourner un Pydantic schema valide — pas de JSON cassé.

Niveau supérieur : Fine-tuner un petit modèle (Phi-3, Mistral 7B) sur tes domaines métier pour extraction locale sans API.

8. Feuille de Route d'Implémentation

Phase 1 — Fondations (Mois 1-3)

- NestJS : auth JWT, RBAC Guards, structure multi-tenant
- Flutter : BDUI renderer JSON → widgets, navigation dynamique
- PostgreSQL : schéma multi-tenant, RLS, pgvector
- Redis : sessions, cache
- CASL : ABAC basique (département, rôle)
- Workflow DAG : validation et exécution des étapes

Phase 2 — IA Intégrée (Mois 4-6)

- FastAPI : microservice LLM, streaming SSE
- LlamaIndex : RAG sur données métier avec pgvector
- Hybrid search : vecteur + keyword + reranking
- Mem0 : mémoire utilisateur long terme
- Chat de configuration : extraction entités avec Instructor
- Génération FSM (XState) depuis config conversationnelle

Phase 3 — Robustesse (Mois 7-9)

- CRDT : synchronisation offline avancée
- Rete Algorithm : moteur de règles ABAC optimisé
- Docling : ingestion de documents existants du client
- Observabilité : Langfuse pour traçage LLM
- Fine-tuning : modèle léger pour extraction métier locale
- Ollama : déploiement LLM local pour clients sans connexion

9. Résumé — Ce qui te différencie

ERP classique	Ton ERP IA UI-Driven
Config : 3-6 mois	Config : 45 min de chat IA
UI fixe pour tous	UI générée par rôle et contexte
RBAC simple	RBAC + ABAC + RLS multi-couche
Offline basique	Offline CRDT + sync intelligente
RAG basique ou absent	Hybrid search + reranking
Règles codées en dur	Moteur de règles Rete dynamique
Workflows figés	Workflows DAG générés par l'IA

Principe directeur

La puissance de cet ERP ne vient pas d'une seule technologie, mais de leur combinaison : BDUI + Config IA + RBAC/ABAC + RAG hybride + Workflows DAG + CRDT offline. Chaque couche renforce les autres. C'est l'intégration cohérente qui crée la différence.

10. UX Adaptative — Ce que l'IA contrôle vraiment

10.1 Règle fondamentale : l'IA propose, l'utilisateur dispose

L'adaptation temps réel de l'UI pendant l'utilisation est une mauvaise idée. Les utilisateurs ont besoin de prévisibilité. Si l'UI bouge pendant qu'ils travaillent, ils perdent leurs repères et font des erreurs.

Ce que l'IA NE fait PAS	Ce que l'IA FAIT
Réorganiser l'UI en cours d'utilisation	Configurer l'UI une fois, au départ
Changer les couleurs ou positions	Choisir les composants et leur contenu
Décider les permissions	Générer les règles visible_if dans la config
Adapter en temps réel sur mobile offline	Générer une config téléchargée une fois
Inventer des composants UI	Assembler depuis le catalogue prédéfini

10.2 L'IA fait les 'if' une seule fois

Zero if métier dans le code Flutter. Toute la logique conditionnelle est générée par l'IA lors de la configuration, stockée en JSON, et téléchargée sur l'appareil. Flutter est un moteur de rendu qui lit des règles — il ne contient aucune logique business.

```
// ❌ MAUVAIS — if métier dans Flutter
if (user.role == 'MANAGER' && commande.montant > 500000) {
  showValidationButton();
}

// ✅ BON — Flutter lit la règle depuis la config locale
// La config a été générée par l'IA lors de l'onboarding
{
  "component": "Button",
  "label": "Valider",
  "visible_if": {
    "AND": [
      { "role": ["MANAGER", "DG"] },
      { "field": "montant", "operator": ">", "value": 500000 }
    ]
  }
}

// Flutter évalue visible_if depuis le JSON — aucune connaissance du métier
```

10.3 Flux offline — la config EST l'app

Phase 1 – EN LIGNE (une seule fois, onboarding)
Chat IA → Config générée → Téléchargée → Stockée Drift/Isar

Phase 2 – OFFLINE (quotidien, sans réseau)

- Flutter lit config locale
- Rend l'UI selon les règles JSON
- RBAC/ABAC appliqué localement
- L'IA n'est plus impliquée
- Zéro appel réseau pour l'UI

Phase 3 – SYNC (quand connexion disponible)

- Détection nouvelle version config (ex: v1.2 → v1.3)
- Télécharge uniquement le diff
- Applique au prochain démarrage
- Pas de mise à jour App Store nécessaire

11. Design System Fermé — La beauté garantie

L'IA ne dessine pas. Elle assemble des composants prédéfinis qui sont déjà beaux. C'est la différence entre un chef qui cuisine et un chef qui choisit sur un menu. La beauté est dans les composants, pas dans les décisions de l'IA.

11.1 Les 3 couches du Design System

Couche 1 — Design Tokens (jamais touchés par l'IA)

Couleurs : primary, success, danger, surface, text — définis une fois, gravés dans le marbre

Espacement : grille de 4px — [4, 8, 12, 16, 24, 32, 48, 64]

Typographie : h1, h2, body, caption — tailles et graisses fixes

Border radius : [4, 8, 16, 999] — pill = 999

L'IA ne mentionne jamais une couleur, une taille ou une police dans sa config.

Couche 2 — Catalogue de composants (fermé)

Affichage : KPICard, StatCard, AlertBanner, SectionHeader, EmptyState, AvatarGroup

Actions : PrimaryButton, SecondaryButton, DangerButton, FAB, IconButton, DropdownMenu

Données : DataTable, ListTile, TimelineCard, ChartBar, ChartPie, MapView

L'IA choisit UNIQUEMENT parmi ces composants. Si une demande n'existe pas dans le catalogue, elle choisit le plus proche.

Instructor (structured outputs) garantit que le JSON retourné est valide — impossible de sortir du catalogue.

Couche 3 — Règles de layout (grammaire de l'UI)

Dashboard → max 4 KPICards en haut, puis 1-2 sections, jamais 3 colonnes sur mobile

Liste → SectionHeader obligatoire, ListTile OU DataTable (pas les deux), FAB si création possible

Formulaire → champs groupés par section, max 5 champs/section, bouton submit toujours en bas

L'IA assemble selon ces règles. Un layout invalide est rejeté par le validateur avant d'être sauvegardé.

11.2 Contraindre l'IA au catalogue

```
// Prompt système de l'Agent Config
Tu es un configurateur d'ERP.
Tu DOIS choisir uniquement parmi ces composants :
[KPICard, StatCard, DataTable, ListTile, AlertBanner,
```

```
PrimaryButton, FAB, ChartBar, ChartPie, ...]
```

Tu DOIS respecter ces règles de layout :

- Dashboard : max 4 KPIs en haut
- Mobile : jamais plus de 2 colonnes
- Formulaire : max 5 champs par section

Tu ne définis JAMAIS : couleurs, tailles, polices, espacements.

Tu retournes UNIQUEMENT du JSON valide selon ce Pydantic schema.

Si une demande sort du catalogue, choisis le composant le plus proche.

12. UX par Métier & Multi-Business

12.1 UX Profiles par secteur

Chaque secteur métier a ses patterns UX validés — conçus par toi une fois, jamais réinventés par l'IA. L'IA choisit parmi ces profiles, elle ne crée pas de nouveaux patterns.

```

catalog/
├── pharmacie/
│   ├── layout_rules.json      ← règles spécifiques pharma
│   ├── screen_templates.json ← écrans types (caisse, stock, ordo)
│   └── ux_patterns.json      ← workflows validés
├── btp/
├── cabinet_medical/
├── commerce_general/
├── restauration/
└── _base/                    ← règles communes à tous les métiers
  
```

12.2 Exemple — UX Profile Pharmacie

```

// pharmacie/ux_patterns.json
{
  "metier": "pharmacie",
  "patterns": {
    "liste_produits": {
      "composant_primaire": "DataTable",
      "colonnes_defaut": ["DCI", "nom_commercial", "stock", "prix"],
      "alerte_critique": "stock < seuil_min",
      "action_principale": "nouvelle_vente"
    },
    "dashboard": {
      "kpis_prioritaires": ["ventes_jour", "ruptures_stock",
"ordonnances_attente"],
      "layout": "kpi_top_list_bottom"
    },
    "workflow_vente": {
      "sequence": ["scan_ordonnance", "verif_stock", "preparation",
"encaissement"],
      "affichage": "stepper_horizontal"
    }
  }
}
  
```

12.3 Gestion Multi-Business

Un client qui gère plusieurs activités (ex: Pharmacie + Cabinet Médical) a 3 options selon ses besoins :

Option	Description	Quand l'utiliser
Espaces séparés	Chaque business charge son UX Profile complet. L'utilisateur switche consciemment.	Métiers très différents, équipes séparées
Fusion par rôle	Le DG voit une vue consolidée. Le comptable voit les finances des 2. L'UI reste celle du rôle.	Même direction, reporting unifié
Modules partagés	L'IA détecte les modules communs (facturation, stock) et propose une gestion unifiée.	Modules réellement identiques entre les 2 métiers

12.4 Règles de fusion multi-métier

4 règles non négociables

Règle 1 — Séparation par défaut : 2 métiers = 2 espaces distincts sauf demande explicite.

Règle 2 — Fusion uniquement sur modules identiques : Stock pharmacie ≠ Stock BTP (unités différentes). Facturation pharmacie ≈ Facturation cabinet (fusionnable).

Règle 3 — Le rôle prime sur le métier : Un DG multi-business a toujours l'UX 'direction', un comptable toujours l'UX 'finance'.

Règle 4 — Jamais de pattern hybride inventé : si 2 métiers n'ont pas de pattern de fusion dans le catalogue → espaces séparés obligatoirement.

12.5 Ce que l'Agent UX fait concrètement

```

Étape 1 : Détecte le(s) métier(s) du client
→ "Pharmacie + Cabinet Médical"

Étape 2 : Charge les UX Profiles correspondants
→ pharmacie/ux_patterns.json
→ cabinet_medical/ux_patterns.json

Étape 3 : Détecte les modules partagés
→ Facturation : présent dans les 2 → propose fusion
→ Stock : unités différentes → séparé
→ Patients/Clients : même entité → propose fusion

Étape 4 : Applique les règles de fusion (jamais d'invention)

Étape 5 : Génère la config JSON finale par espace
→ config_pharmacie.json
→ config_cabinet.json

```

→ `config_shared_modules.json`

⚠ Ne sort JAMAIS du catalogue de composants et de patterns

Approche	Résultat UX	Verdict
Agent UX complètement libre	UI imprévisible, bizarre, impossible à tester	✗ Refusé
UX 100% fixe sans IA	Rigide, ne s'adapte pas au métier	✗ Insuffisant
Agent UX contraint par catalogue métier	Belle, cohérente, adaptée au secteur	✓ Adopté
Multi-business fusionné naïvement	UI hybride incohérente	✗ Refusé
Multi-business par espaces + fusion modules	Propre, maintenable, prévisible	✓ Adopté

14. Architecture NestJS — Structure & Flux

14.1 Structure des modules

src/	
├─ core/	← Chargé une fois, global
│ ├── auth/	JWT, OAuth2, sessions
│ ├── rbac/	Guards, Decorators, Roles
│ ├── abac/	CASL abilities, règles dynamiques
│ ├── tenant/	Résolution du tenant par requête
│ ├── audit/	Log chaque action sensible
│ └─ config/	Variables d'environnement typées
├─ bdui/	← Génère le JSON UI par tenant/rôle
│ ├── bdui.service.ts	Charge config + applique ABAC
│ ├── bdui.resolver.ts	Résout les data sources du JSON
│ └─ bdui.validator.ts	Valide le JSON avant sauvegarde
├─ config-agent/	← Onboarding conversationnel
│ ├── chat.gateway.ts	WebSocket + SSE vers Flutter
│ ├── extraction.service.ts	LLM → entités métier structurées
│ ├── workflow.validator.ts	DAG validation des workflows générés
│ └─ tenant-generator.ts	Génère la config JSON du tenant
├─ modules/	← Modules ERP génériques
│ ├── commandes/	
│ ├── stock/	
│ ├── facturation/	
│ ├── rh/	
│ └─ rapports/	
└─ ai/	← Pont vers FastAPI
│ ├── ai.client.ts	HTTP client vers FastAPI
│ ├── streaming.service.ts	SSE relay FastAPI → Flutter
│ └─ context.service.ts	Prépare le contexte filtré ABAC

14.2 Flux d'une requête — 6 couches

Flutter envoie : GET /api/commandes
1. TenantMiddleware
→ lit le header X-Tenant-ID
→ charge la config tenant depuis Redis (cache)
→ injecte dans le contexte de la requête
2. JwtGuard
→ vérifie le token
→ extrait user + roles + attributes
3. RbacGuard

- @Roles('MANAGER', 'VENDEUR')
 - ce rôle a-t-il accès à cette route ?
4. AbacInterceptor
 - CASL : ability.can('read', 'Commande')
 - construit le filtre SQL selon les attributs
 - { departementId: user.deptId, montant: { lte: user.plafond } }
 5. CommandesService
 - applique le filtre ABAC sur la query TypeORM
 - JAMAIS de query sans filtre tenant + abac
 6. AuditInterceptor
 - log : qui, quoi, quand, résultat
 - stocké en base pour traçabilité complète

14.3 BDUI Service — génération du JSON par rôle

```
// bdui.service.ts
@Injectable()
export class BDUIService {
  async getScreen(tenantId: string, userId: string, screenId: string) {
    // 1. Charge la config tenant depuis Redis ou DB
    const config = await this.configCache.get(tenantId);

    // 2. Résout l'écran demandé
    const screen = config.screens[screenId];

    // 3. Filtre les composants selon ABAC
    const user = await this.userService.getWithContext(userId);
    const filtered = this.filterComponents(screen, user);

    // 4. Résout les data sources (valeurs réelles)
    const resolved = await this.resolver.resolve(filtered, user);

    return resolved; // JSON prêt pour Flutter
  }

  private filterComponents(screen: Screen, user: UserContext) {
    return screen.zones.map(zone => ({
      ...zone,
      components: zone.components.filter(c =>
        this.evaluator.evaluate(c.visible_if, user)
      )
    }));
  }
}
```

15. Database Schema — PostgreSQL Multi-Tenant

15.1 Approche : Schema-per-Tenant

Chaque client a son propre schema PostgreSQL. L'isolation est totale — pas de colonne `tenant_id` partout, backup par client possible, RLS simplifié. Le schema public contient uniquement les tables partagées.

Schema public (partagé)	Schema {tenant_id} (par client)
public.tenants	{tenant}.commandes
public.tenant_configs (JSONB)	{tenant}.stock
public.users	{tenant}.factures
public.audit_logs	{tenant}.clients_erp
public.embeddings_index	{tenant}.embeddings (pgvector)
public.workflow_runs	{tenant}.workflows_history

15.2 Table `tenant_configs` — le coeur du BDUI

```
CREATE TABLE public.tenant_configs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL REFERENCES public.tenants(id),
  version     TEXT NOT NULL DEFAULT '1.0.0',
  metier      JSONB NOT NULL,    -- ['pharmacie', 'btp']
  modules     JSONB NOT NULL,    -- modules activés
  roles       JSONB NOT NULL,    -- définitions RBAC/ABAC
  ux_profiles JSONB NOT NULL,    -- règles UI par rôle
  screens     JSONB NOT NULL,    -- config BDUI des écrans
  workflows   JSONB NOT NULL,    -- FSM des workflows
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  is_active   BOOLEAN DEFAULT TRUE
);

-- Index pour le diff de config (sync offline)
CREATE INDEX idx_tenant_config_version
  ON tenant_configs(tenant_id, version);

-- Une seule config active par tenant
CREATE UNIQUE INDEX idx_tenant_active_config
  ON tenant_configs(tenant_id) WHERE is_active = TRUE;
```

15.3 pgvector — RAG par tenant

```
-- Table embeddings dans chaque schema tenant
CREATE TABLE {tenant}.embeddings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content     TEXT NOT NULL,
  metadata    JSONB,           -- { module, entite, dept_id, roles }
  embedding   vector(1536),    -- OpenAI / Cohere / Claude embeddings
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Index IVFFlat pour recherche approximative rapide
CREATE INDEX ON {tenant}.embeddings
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Index Full-Text pour le BM25 (hybrid search)
CREATE INDEX ON {tenant}.embeddings
  USING gin(to_tsvector('french', content));
```

15.4 Row Level Security — dernière ligne de défense

```
-- Activer RLS sur toutes les tables tenant
ALTER TABLE {tenant}.commandes ENABLE ROW LEVEL SECURITY;

-- Policy : chaque user voit seulement son département
-- sauf DG et ADMIN qui voient tout
CREATE POLICY commandes_dept_policy ON {tenant}.commandes
  USING (
    departement_id = current_setting('app.user_dept_id')::uuid
    OR current_setting('app.user_role') IN ('DG', 'ADMIN')
  );

-- NestJS injecte le contexte user avant chaque query
await this.entityManager.query(
  `SET LOCAL app.user_dept_id = '${user.deptId}';
   SET LOCAL app.user_role = '${user.role}';`
);
-- Même si NestJS est bypassé, la DB protège
```

15.5 Gestion de la config offline — versioning

```
-- L'app Flutter vérifie si sa config est à jour
GET /api/config/version → { version: '1.3.0', hash: 'abc123' }

-- Si différent de la version locale :
GET /api/config/diff?from=1.2.0&to=1.3.0
```

```
→ { changed: ['screens.commandes', 'roles.VENDEUR'], ... }

-- Flutter applique uniquement le diff
-- Pas besoin de retélécharger toute la config

-- Historique des configs (rollback possible)
CREATE TABLE public.tenant_config_history (
  id          UUID PRIMARY KEY,
  tenant_id   UUID NOT NULL,
  version     TEXT NOT NULL,
  config      JSONB NOT NULL,
  changed_by  UUID NOT NULL,
  changed_at  TIMESTAMPTZ DEFAULT NOW(),
  diff        JSONB          -- ce qui a changé vs version précédente
);
```

16. FastAPI — Microservice IA

16.1 Structure du service

```

ai-service/
├── main.py
├── routers/
│   ├── chat.py           SSE streaming vers NestJS
│   ├── rag.py            Hybrid search + reranking
│   ├── config_agent.py   Extraction config depuis conversation
│   └── embeddings.py     Génération + indexation
├── agents/
│   ├── config_agent.py   LangChain agent onboarding
│   ├── ux_agent.py       Sélection UX Profile + validation
│   └── query_agent.py    Réponses aux questions métier
├── rag/
│   ├── hybrid_search.py  vecteur + BM25 + reranking
│   ├── indexer.py        Docling → chunks → embeddings
│   └── memory.py         Mem0 intégration
└── schemas/
    ├── bdui_config.py    Pydantic schema — JSON config ERP
    └── extraction.py     Entités extraites de la conversation
  
```

16.2 SSE Streaming — token par token vers Flutter

```

@router.post('/chat/stream')
async def chat_stream(request: ChatRequest):
    async def generate():
        # Contexte déjà filtré par ABAC côté NestJS
        # FastAPI ne reçoit QUE les données autorisées
        filtered_context = request.pre_filtered_context

        memories = await mem0.search(
            query=request.messages[-1].content,
            user_id=request.user_id,
            limit=5
        )

        rag_results = await hybrid_search(
            query=request.messages[-1].content,
            user_ctx=request.user_context
        )

        async for chunk in claude_client.stream(
            messages=request.messages,
            system=build_prompt(filtered_context, memories, rag_results),
            max_tokens=1000
  
```

```

    ):
        yield f'data: {chunk.text}\n\n'

    yield 'data: [DONE]\n\n'

    return StreamingResponse(
        generate(),
        media_type='text/event-stream'
    )

```

16.3 Config Agent — extraction structurée avec Instructor

```

import instructor
from anthropic import Anthropic
from pydantic import BaseModel

class ERPConfig(BaseModel):
    metiers: List[str]          # doit être dans METIERS_CATALOGUE
    modules: List[Module]
    roles: List[Role]
    workflows: List[Workflow]   # validés comme DAG
    abac_rules: List[ABACRule]
    ux_profile: str             # doit exister dans le catalogue

class ConfigAgent:
    def __init__(self):
        self.client = instructor.from_anthropic(Anthropic())

    async def extract_config(self, conversation) -> ERPConfig:
        return self.client.messages.create(
            model='claude-sonnet-4-20250514',
            max_tokens=4000,
            messages=conversation,
            system=f'''
                Tu extrais la configuration ERP depuis la conversation.
                Métiers disponibles UNIQUEMENT : {METIERS_CATALOGUE}
                UX profiles disponibles UNIQUEMENT : {UX_PROFILES}
                Valide que tous les workflows sont des DAG valides.
                Retourne UNIQUEMENT le schema ERPConfig.
            '''
        ),
        response_model=ERPConfig, # Instructor garantit le schema
    )
    # Si le JSON est invalide → Instructor relance automatiquement
    # Impossible de retourner un composant hors catalogue

```

17. Moteur IA — RAG Hybride, Mémoire & Agents

17.1 RAG Hybride — au-delà du RAG basique

Le RAG basique (embedding seul) rate 30-40% des résultats pertinents. Le hybrid search combine vectoriel + keyword, puis un reranker cross-encoder trie les résultats. Le filtre ABAC s'applique AVANT la recherche — le LLM ne voit jamais de données non autorisées.

```

async def hybrid_search(query: str, user_ctx: UserContext):
    # 1. Embedding de la query
    embedding = await embed(query)

    # 2. Filtre ABAC – appliqué AVANT la recherche
    abac_filter = {
        'departement_id': user_ctx.dept_id,
        'visible_roles': {'$in': user_ctx.roles}
    }

    # 3. Recherche vectorielle (sémantique) + keyword (BM25)
    vector_results, keyword_results = await asyncio.gather(
        pgvector.query(embedding, filter=abac_filter, limit=20),
        pg_fulltext.search(query, filter=abac_filter, limit=20)
    )

    # 4. Fusion – Reciprocal Rank Fusion
    merged = reciprocal_rank_fusion(
        vector_results, keyword_results, k=60
    )

    # 5. Reranking – Cross-encoder (30-40% de pertinence en plus)
    reranked = await cohere_rerank(
        query=query, documents=merged, top_n=5
    )

    return reranked # 5 résultats ultra-pertinents pour le LLM

```

17.2 Mémoire utilisateur — Mem0

Mem0 donne au LLM une mémoire persistante cross-sessions. L'ERP se souvient des habitudes de chaque utilisateur : modules préférés, formats d'export, patterns de recherche fréquents.

```

from mem0 import Memory
memory = Memory()

# Après chaque interaction – sauvegarder
async def save_interaction(user_id: str, interaction: dict):

```

```

memory.add(
    messages=interaction['messages'],
    user_id=user_id,
    metadata={
        'tenant_id': interaction['tenant_id'],
        'module': interaction['module']
    }
)
# Mem0 extrait automatiquement les faits importants :
# 'L'utilisateur cherche souvent les commandes du mois'
# 'Il préfère les exports en XLS'
# 'Il travaille principalement sur le département RH'

# Avant chaque appel LLM – récupérer le contexte
async def get_user_context(user_id: str, query: str):
    memories = memory.search(
        query=query, user_id=user_id, limit=5
    )
    return memories # injecté dans le system prompt

```

17.3 Indexation des documents métier — Docling

```

from docling.document_converter import DocumentConverter

async def index_document(file_path: str, tenant_id: str, metadata: dict):
    # 1. Extraction du contenu (PDF, Excel, Word)
    converter = DocumentConverter()
    doc = converter.convert(file_path)
    text = doc.document.export_to_markdown()

    # 2. Chunking intelligent (respecte la structure du doc)
    chunks = smart_chunk(text, max_tokens=512, overlap=50)

    # 3. Embedding de chaque chunk
    embeddings = await embed_batch(chunks)

    # 4. Stockage dans le schema tenant avec métadonnées ABAC
    for chunk, embedding in zip(chunks, embeddings):
        await db.execute(f'''
            INSERT INTO {tenant_id}.embeddings
            (content, embedding, metadata)
            VALUES ($1, $2, $3)
        ''', chunk, embedding, {
            **metadata,
            'dept_id': metadata.get('dept_id'),
            'visible_roles': metadata.get('roles', ['ADMIN'])
        })

```

17.4 Vue d'ensemble — flux complet IA

```
Flutter (user pose une question)
  ↓ WebSocket
NestJS
  → Vérifie RBAC/ABAC
  → Construit user_context filtré (dept, rôles, plafonds)
  → Relais à FastAPI avec UNIQUEMENT les données autorisées
  ↓ HTTP
FastAPI
  → Mem0 : récupère mémoire utilisateur (habitudes, préférences)
  → Hybrid Search : vecteur + BM25 → RRF → Reranking
    (filtré par ABAC avant même la recherche)
  → Construit le prompt avec contexte + mémoire + RAG
  → Appelle Claude API en streaming
  ↓ SSE token par token
NestJS → relay SSE → Flutter
  → Audit log de l'interaction
  → Mem0 : sauvegarde la nouvelle interaction
```

17.5 Résumé des choix techniques IA

Besoin	Outil choisi	Pourquoi
RAG vectoriel	pgvector (PostgreSQL)	Pas de DB supplémentaire, filtre ABAC natif SQL
Keyword search	pg_trgm + tsvector	Intégré PostgreSQL, BM25-like
Reranking	Cohere Rerank	Cross-encoder, +30-40% pertinence
Mémoire long terme	Mem0	Extraction automatique des faits importants
Extraction structurée	Instructor + Pydantic	JSON toujours valide, impossible de sortir du catalogue
Parsing documents	Docling (IBM)	Excellent sur PDF, tableaux, Excel
Orchestration agents	LangChain	Chains, agents, outils personnalisés
Streaming	FastAPI + SSE	Token par token, async natif Python
LLM principal	Claude API (Sonnet)	Meilleur rapport qualité/coût pour ERP
LLM offline	Ollama + Mistral 7B	Local, sans réseau, pour clients isolés

18. Modules ERP — Approche & Création

18.1 Le vrai problème des modules

ERP classique	Ton ERP BDUI
50 modules codés en dur	6 moteurs génériques + configs JSON
Chaque client = les mêmes modules	Chaque client adapte les modules standards
Personnalisation = modifier le code	Personnalisation = surcharger le JSON
Nouveau métier = nouveau dev	Nouveau métier = nouveau fichier JSON
50 écrans Flutter codés	1 moteur Flutter qui rend tout

18.2 Les 2 types de modules

Type 1 — Moteurs génériques (tu codes une fois)

ModuleList → gère n'importe quelle liste d'entités
 ModuleForm → gère n'importe quel formulaire de création/édition
 ModuleDetail → gère n'importe quelle fiche détail
 ModuleReport → gère n'importe quel rapport/export
 ModuleKanban → gère n'importe quel workflow en colonnes
 ModuleDashboard → agrège des KPIs de n'importe quelle source
 Ces 6 moteurs couvrent 100% des besoins d'affichage d'un ERP.

Type 2 — Modules métier (déclarés en JSON, pas codés)

Commandes, Stock, Factures, RH, Livraisons... ne sont pas du code.
 Ce sont des fichiers JSON qui déclarent quel moteur utiliser, quelles colonnes afficher, quels workflows déclencher.
 Un nouveau module métier = un nouveau fichier JSON dans le catalogue.
 Zéro nouveau code Flutter ou NestJS nécessaire.

18.3 Catalogue de modules standards

```

catalog/modules/
├─ gestion/
│   ├─ commandes.json      (ModuleList + workflow approval)
│   └─ stock.json          (ModuleList + alertes seuils)

```

├─ clients.json	(ModuleList + fiche détail)
└─ fournisseurs.json	(ModuleList + fiche détail)
├─ finance/	
│ └─ factures.json	(ModuleList + génération PDF)
│ └─ paiements.json	(ModuleList + réconciliation)
│ └─ rapports_fin.json	(ModuleDashboard + export)
├─ rh/	
│ └─ employees.json	(ModuleList + fiche)
│ └─ conges.json	(ModuleKanban + workflow)
│ └─ paie.json	(ModuleReport + calculs)
├─ operations/	
│ └─ livraisons.json	(ModuleList + géoloc)
│ └─ planning.json	(ModuleKanban)
│ └─ chantiers.json	(ModuleDashboard)

18.4 Anatomie d'un module standard

```
// catalog/modules/gestion/commandes.json
{
  "module_id": "commandes",
  "engine": "ModuleList",
  "entity": "commande",
  "label": "Commandes",
  "icon": "cart",
  "schema": {
    "ref": { "type": "string", "label": "Référence", "required": true },
    "client": { "type": "relation", "target": "client", "label": "Client" },
    "montant": { "type": "currency", "label": "Montant" },
    "statut": { "type": "enum", "values":
["brouillon","soumise","validee","livree"] }
  },
  "default_columns": ["ref", "client", "montant", "statut"],
  "default_filters": ["statut", "date_range"],
  "workflow": "commande_workflow",
  "actions": {
    "create": { "roles": ["VENDEUR", "MANAGER"] },
    "approve": { "roles": ["MANAGER", "DG"], "condition": "montant > 0" },
    "export": { "roles": ["COMPTABLE", "DG"] }
  }
}
```

18.5 Comment un client adapte un module standard

Chaque client hérite d'un module standard et surcharge uniquement ce qui diffère. L'IA génère ces overrides depuis la conversation de configuration — zéro code écrit.

```
// Config tenant Pharmacie Kossyam – override du module commandes
{
  "module_id": "commandes",
  "inherits": "catalog/modules/gestion/commandes",
  "overrides": {
    "label": "Bons de commande",
    "schema": {
      "ordonnance": { "type": "file", "label": "Ordonnance", "required": true }
    },
    "default_columns": ["ref", "client", "ordonnance", "montant", "statut"],
    "actions": {
      "approve": { "condition": "montant > 100000", "roles": ["PHARMACIEN"] }
    }
  }
}

// Ce que l'IA a compris depuis la conversation :
// 'chaque commande doit avoir une ordonnance'
// → ajouter champ ordonnance (type: file, required: true)
// 'validée par le pharmacien si > 100k'
// → override action approve : condition + rôle
```

18.6 ModuleList — le moteur générique Flutter

```
// Ce composant ne sait pas ce qu'est une 'Commande'
// Il sait afficher N'IMPORTE QUELLE liste
class ModuleList extends StatelessWidget {
  final ModuleConfig config; // vient du JSON
  final UserContext user;

  Widget build(BuildContext context) {
    return Column(children: [
      // Header avec titre + actions filtrées par ABAC
      ModuleHeader(
        title: config.label,
        actions: config.actions
          .where((a) => evaluator.canDo(a, user))
          .map((a) => ActionButton(action: a))
          .toList()
      ),

      // Filtres générés depuis config.filters
      if (config.filters != null)
        FilterBar(filters: config.filters!),

      // Tableau généré depuis config.columns
      DynamicDataTable(
        columns: config.columns,
        source: config.dataSource,
        user: user
      ),
    ],
  )
}
```

```

        // FAB si l'user peut créer
        if (evaluator.can('create', config.entity, user))
            ERPFab(action: 'create_${config.entity}')
    });
}
}

```

18.7 Backend — endpoint générique pour tous les modules

```

// NestJS — un seul endpoint pour tous les modules
@Get('/:moduleId/data')
async getModuleData(
    @Param('moduleId') moduleId: string,
    @CurrentUser() user: UserContext,
    @TenantConfig() config: TenantConfig
) {
    // Charge la définition depuis la config tenant (héritée + overrides)
    const moduleDef = config.modules[moduleId];

    // Résout la query selon le schema + filtre ABAC
    return this.moduleEngine.query({
        entity: moduleDef.entity,
        schema: moduleDef.resolvedSchema, // standard + overrides fusionnés
        filters: this.buildAbacFilters(moduleDef, user),
        columns: moduleDef.columns
    });
}

// Un seul endpoint pour créer dans n'importe quel module
@Post('/:moduleId/create')
async createEntity(
    @Param('moduleId') moduleId: string,
    @Body() data: Record<string, any>,
    @CurrentUser() user: UserContext
) {
    const moduleDef = config.modules[moduleId];
    await this.schemaValidator.validate(data, moduleDef.schema);
    await this.workflowEngine.trigger('create', moduleDef, data, user);
}

```

18.8 Ce que l'IA fait lors de la config des modules

```

User : 'On gère des bons de commande,
       chaque commande doit avoir une ordonnance
       et être validée par le pharmacien si > 100k'

```

```
Config Agent (LangChain + Instructor) :  
1. Détecte : module 'commandes' dans le catalogue  
2. Identifie les overrides :  
   - label → 'Bons de commande'  
   - Ajouter champ 'ordonnance' (type: file, required: true)  
   - Action approve : condition montant > 100000, rôle PHARMACIEN  
3. Génère le JSON override (validé par Pydantic schema)  
4. Vérifie cohérence avec le reste de la config  
5. Sauvegarde dans tenant_configs
```

Résultat : module opérationnel sans une ligne de code écrite

18.9 Résumé — ce que tu crées vs ce que tu declares

Élément	Comment	Nombre
Moteurs Flutter (ModuleList, Form...)	Codé une fois	6 moteurs max
Modules standards catalogue	Fichiers JSON	~20 modules
Modules spécifiques client	JSON override généré par l'IA	Illimité
Endpoint NestJS modules	Générique, codé une fois	2 endpoints
Nouveau type de métier	Nouveau fichier JSON catalogue	Zéro code
Personnalisation client	Override JSON via chat IA	Zéro code

19. NestJS Seul vs Supabase — Décision Argumentée

19.1 Ce que Supabase apporte réellement

Supabase self-hosted c'est un ensemble de ~8 services Docker : PostgreSQL, GoTrue (Auth), Realtime (Elixir), Storage (Go), Kong (API Gateway), Studio (Dashboard), PostgREST (API auto). Voyons ce qui est utile vs redondant avec NestJS.

Service Supabase	Équivalent NestJS	Verdict
GoTrue (Auth)	Passport.js + @nestjs/jwt	✗ Redondant
Realtime (WebSocket)	NestJS WebSocket Gateway	✗ Redondant
PostgREST (API auto)	NestJS controllers	✗ Inutile
Kong (API Gateway)	NestJS lui-même	✗ Redondant
Storage (fichiers)	MinIO (1 service Docker)	✗ MinIO suffit
Studio (Dashboard DB)	Adminer (1 service léger)	⚠ Adminer suffit en dev
PostgreSQL + pgvector	PostgreSQL + pgvector	✓ Même chose
RLS policies	RLS policies SQL	✓ Même chose

19.2 Supabase self-hosted — le vrai coût

```
Supabase self-hosted = 8 services Docker :
  postgres    → la DB
  gotrue      → auth (déjà dans NestJS)
  realtime    → websocket (déjà dans NestJS)
  storage-api → fichiers (MinIO = 1 service)
  imgproxy    → resize images
  kong        → API gateway (NestJS = gateway)
  studio      → dashboard (Adminer = 50MB)
  meta        → metadata service

RAM minimale recommandée : 4-8 GB juste pour Supabase
Complexité opérationnelle : haute (8 services à maintenir)
Valeur ajoutée réelle pour ton ERP : faible

MinIO + Adminer = 2 services, <500MB RAM, même fonctionnalité
```

19.3 Stack NestJS seul — ce qu'elle couvre

Besoin	Solution dans NestJS seul	Complexité
Auth JWT + Refresh	Passport.js + @nestjs/jwt	Faible

OAuth2 (Google...)	Passport strategies	Faible
WebSocket / Realtime	@nestjs/websockets Gateway	Faible
Storage fichiers	MinIO via @aws-sdk/client-s3	Faible
Dashboard DB (dev)	Adminer Docker (50MB)	Nulle
RBAC Guards	@nestjs/passport + decorators	Moyenne
ABAC complexe	CASL + Casbin	Moyenne
Multi-tenant	Middleware + schema switching	Moyenne
Migrations DB	Prisma Migrate	Faible
Audit logs	NestJS Interceptor	Faible

19.4 Docker Compose final — 5 services seulement

```

version: '3.8'
services:

  nestjs:
    build: ./apps/api
    ports: ['3000:3000']
    depends_on: [postgresql, redis, minio]
    environment:
      DATABASE_URL: postgresql://...
      REDIS_URL: redis://redis:6379
      MINIO_ENDPOINT: minio:9000
      JWT_SECRET: ${JWT_SECRET}

  fastapi:
    build: ./apps/ai
    ports: ['8000:8000']
    depends_on: [postgresql, redis]
    environment:
      DATABASE_URL: postgresql://...
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}

  postgresql:
    image: pgvector/pgvector:pg16
    volumes: ['postgres_data:/var/lib/postgresql/data']
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:7-alpine
    volumes: ['redis_data:/data']

  minio:
    image: minio/minio

```

```

command: server /data --console-address ':9001'
ports: ['9000:9000', '9001:9001']
volumes: ['minio_data:/data']

# Dev seulement – retirer en production
adminer:
  image: adminer
  ports: ['8080:8080']

```

19.5 MinIO — Storage S3-compatible

MinIO remplace Supabase Storage en 1 service Docker léger. Il est S3-compatible, donc les SDKs AWS fonctionnent directement. Il gère les PDFs, ordonnances, images et tout fichier uploadé par les clients ERP.

```

// NestJS – upload vers MinIO (même API que AWS S3)
@Inject()
export class StorageService {
  private s3 = new S3Client({
    endpoint: process.env.MINIO_ENDPOINT,
    credentials: {
      accessKeyId: process.env.MINIO_ACCESS_KEY,
      secretAccessKey: process.env.MINIO_SECRET_KEY,
    },
    forcePathStyle: true, // requis pour MinIO
  });

  async upload(tenantId: string, file: Express.Multer.File) {
    return this.s3.send(new PutObjectCommand({
      Bucket: tenantId, // bucket isolé par tenant
      Key: `${uuid()}-${file.originalname}`,
      Body: file.buffer,
      ContentType: file.mimetype,
    }));
  }
}

```

19.6 Verdict final

NestJS seul + PostgreSQL + MinIO + Redis	Supabase self-hosted + NestJS
5 services Docker	12+ services Docker
~1-2 GB RAM total	~4-8 GB RAM juste pour Supabase
Contrôle total, stack cohérente	Contrôle total, complexité opérationnelle haute

Auth dans NestJS (Passport)	Auth dupliquée (GoTrue + Passport)
Realtime dans NestJS Gateway	Realtime dupliqué (Elixir + NestJS)
MinIO = 1 service léger	Storage Supabase = service lourd
Migration PostgreSQL facile	Migration depuis Supabase complexe
✓ CHOIX RETENU	✗ Écarté — complexité sans valeur ajoutée

13. Résumé Final — Vision Complète

Ce qui rend cet ERP unique

1. Configuration en 45 min via chat IA — pas 6 mois de paramétrage
2. BDUI Engine — ComponentRegistry + RuleEvaluator + LayoutResolver, zéro if métier dans Flutter
3. shadcn_ui Flutter + ShadCN React — même langage visuel mobile et web, cohérence totale
4. Widgetbook — maquettage vivant par composants, pas par écrans
5. Design System fermé — l'IA assemble depuis le catalogue, elle n'invente pas
6. UX Profiles par secteur — belle et cohérente pour chaque métier
7. Modules = 6 moteurs génériques + JSONs — zéro code pour un nouveau module ou client
8. Override par héritage — chaque client surcharge uniquement ce qui diffère du standard
9. Stack 5 services seulement — NestJS + FastAPI + PostgreSQL + Redis + MinIO, pas de Supabase
10. NestJS 6 couches — Tenant → JWT → RBAC → ABAC → Service → Audit, chaque requête protégée
11. PostgreSQL schema-per-tenant — isolation totale, RLS, pgvector, config JSONB versionnée
12. MinIO — Storage S3-compatible en 1 service léger, buckets isolés par tenant
13. FastAPI RAG hybride — vecteur + BM25 + RRF + Reranking, +30-40% pertinence vs RAG basique
14. Instructor + Pydantic — extraction structurée fiable, impossible de sortir du catalogue
15. Mem0 — l'ERP se souvient des habitudes de chaque utilisateur cross-sessions
16. Offline-first — config JSON versionnée téléchargée, sync par diff, zéro réseau pour l'UI
17. RBAC + ABAC + RLS — le LLM ne reçoit que les données déjà filtrées, jamais accès brut
18. Multi-business propre — espaces séparés + fusion sur modules partagés uniquement
19. Algorithmique avancé — DAG, CRDT, Rete, Hybrid RAG, FSM — ce que les autres n'ont pas

— Fin du document —